Adaptive Topology Structures for Improved Efficiency in Federated Learning

Jacob Joseph
University of Texas at Austin

Nick Strohmeyer
University of Texas at Austin

December 05, 2024

Abstract

In this project we are interested in developing a novel way to do Federated Learning (FL) in a dynamic environment with mobile devices. Our goal is to improve upon the centralized FL baseline, by enabling device-to-device communication between rounds of global aggregation. We hope to leverage this and exploit real-time network conditions to enhance performance using fewer global rounds. In our second milestone, we explore several semi-centralized FL schemes to build an understanding on how network topology affects training. We found that our FL schemes outdid star in terms of accuracy, however, we failed to overcome costs of networks.

1 Introduction

Centralized Federated Learning (FL) is a distributed ML training scheme which operates an underlying star topology of individual devices, each of which trains a model on a local dataset, which is sent back to the central server for global aggregation. This is an efficient way to aggregate knowledge, but involves high communication overhead and typically. On the other hand, fully *decentralized* FL is inefficient at training a shared model, as no single device will have a dataset representative of the global distribution and aggregation is more challenging. A growing trend is to use hierarchical (or "semi-centralized") FL schemes which trade off advantages and disadvantages of both extremes.

Many FL algorithms consider *static* network topologies when, in reality, participating devices are mobile. In our project, we aim to leverage device mobility by creating an adaptive hierarchical scheme which uses a top-down approach to adaptively assign an efficient topology inspired by a weighted preferential attachment algorithm [CITE] as well as a bottom-up approach in which mobile devices talk to their neighbors and attempt to organize into communities based on model similarity. While neither component of this approach is novel in and of itself, the novelty lies in our application of these techniques to improve the efficiency of federated learning on an adaptive mobile network.

2 Previous Work

A recent work considered the mobile FL problem and exploited properties of the heterogeneous network conditions and existence of fixed WiFi access points to improve performance [4]. Our approach is related to this, but we look at an even less centralized variant of the problem in which all nodes are mobile devices and aggregation points are selected dynamically. Another work [6] considered different classes of network topologies and their effect on information flow in fully decentralized FL. Our work is inspired by some of their findings but extends their study by considering real-world network conditions. In hierarchical FL, it is a common to aggregate models within communities prior to global aggregation [3]. We will also take this approach and plan to let mobility and network conditions specifically inform community assignment. We have also noted of a previous paper utilizing a form of the weighted preferential attachment for evolving network [1].

3 Approach

Overview: We simulate mobile federated learning in which 100 devices move around Austin, each equipped with a convolution neural network with model weights w_t^i and a unique partition of the CIFAR-10 dataset. To make the setting more realistic, we assume a non-iid distribution of data among devices. Our approach uses a two-step, hierarchical aggregation scheme. At the top level, the cloud trains the global model and assigns a network topology attempting to optimize for communication efficiency. At the local level, devices aggregate over multiple communication rounds at strategically selected "hub" devices, which we refer to as "access points" (AP). We assume the server has knowledge of each device's location and AP models at every round. Devices have no knowledge of each other's datasets but can share model weights with neighboring devices, which incurs a cost of communication proportional to the Euclidean distance between them.

We chose the hierarchical scheme based on its prior success [4]. Given our two-level structure, we aim to improve over the centralized(star) FL baseline at two points: during global rounds when the server assigns initial topology, and during local rounds when devices self-organize into communities to optimize training and address the non-iid distribution challenge. The general template of this 2-level FL approach is outlined in algorithm 1. By combining strategies at both the global and local levels, we hope to reduce total communication cost and accelerate global model training. The remainder of this section details how we achieve these objectives in milestone 2.

3.1 Device Mobility

To simulate device movement, we leverage a real-world mobility dataset (FourSquare) which provides a series of de-

vice locations throughout the Austin metropolitan area from May 2020. Selecting the top 100 most frequently occurring devices from FourSquare, we then stich together 10 days (May 10 to May 20) of hourly data during the peak hours of activity (8am to 7pm) giving us a total of 120 frames which temporally align with global server rounds. It is not uncommon to find frames for which not every device records a location. We deal with these missing frames by interpolating the device's last known location with the next. In this way, we ensure every device participates at each server round and has a known location.

3.2 Building the Network

The FL network consists of the 100 mobile devices (nodes) with new geographical positions at each server round. An edge represents a connection between neighboring devices that share their model weights. Based on the new positions of devices, the server decides an initial topology, where it inserts edges between devices to determine who can communicate with whom, and provides each device with an AP to which it sends its model for local aggregation. How the cloud makes these decisions will be described in detail in the section on algorithms. This describes the semi-centralized/hierarchical case. The fully centralized baseline is trivially represented by a star topology.

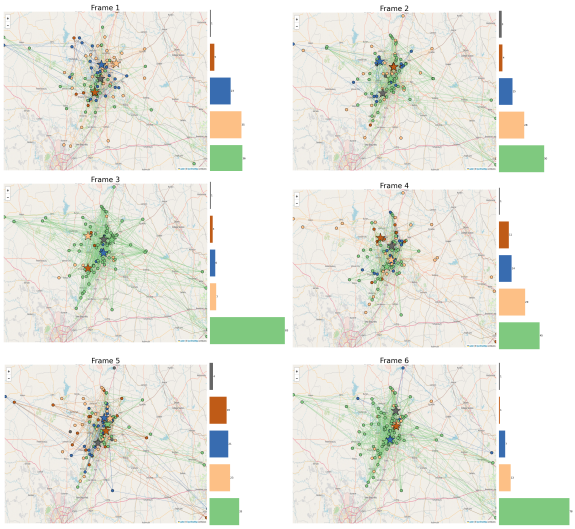

Figure 1: Above is the device positions and network topology, color-coded by community assignment over the first 6 global training rounds for the DPP algorithm. We observe the community assignment shifts between a mode where a few large communities dominate the round to cases where there is a more even distribution of devices. This is driven by the underlying preferential attachment mechanisms, the updating locations, and the cosine similarity of neighboring devices.

3.3 Algorithms

The template of our hierarchical approach is given in algorithm 1. The FL task is broken into $t \in T_S = 35$ total

Algorithm 1 Dynamic Mobile Hierarchical FL

Inputs: Device set, (non-iid) data distribution, random initial weights, coordinates $\mathcal{X}, \mathcal{D}, \mathcal{WP}$
Init: global model η, aggregation function $h(\cdot)$
for each server round **do**
  Move Devices (get $\boldsymbol{p_t}$ from $\mathcal{P}$
  Run Global Linking Algorithm, obtain new $\mathcal{A}_\sqcup$ set, assign $\mathcal{C}_t$
  Pass η to each access point in $\mathcal{A}_t$
  (Optionally) Add self organization step **??** for new $\mathcal{C}_t$
  for each aggregation round **do**
    Distribute models to each device $x_j \in C_t^j$
    Train $w \in W$
    $w_{a_j} \leftarrow h(W_{x \in C_t^j})$ Aggregate weights from community set to each $a_j \in \mathcal{A}_t$
  end for
  $\eta \leftarrow h(\mathcal{A}_t)$ aggregate to global model
end for

server rounds, each consisting of $\tau \in T_A = 2$ local communication rounds. The server trains and evaluates the global model every server round t. Using updated device positions $\mathcal{P}_t$, it runs the global linking/rewiring algorithm to set the new network topology (denoted $\mathcal{G}_t(V, E)$). From $\mathcal{G}_t$, it selects a subset of devices $\mathcal{A}_t V$ to serve as access points (AP). In this milestone and the last, we select the AP's as the top 5 nodes with the *highest betweenness centrality*. Then, the server assigns all other devices to its nearest AP (based on Euclidean distance) thus forming communities C_t^j (where j denotes the j^{th} associated access point $AP_j \in \mathcal{A}_t$). Lastly the cloud sends each AP the global model η and their set community of members. The AP's then distribute the new global model to their member devices algorithm 1.

Each device stores two models, one for its own local training and one which is a copy of its assigned AP's model. We refer to the latter as the *community model*. Optionally, devices can then attempt to re-organize themselves into more optimal community arrangements based on dataset information (see below). Then begins T_A rounds of local training. After each round of local training, devices aggregate at their access point and receive a new community model. After the last communication round, the APs send their models back to the cloud for global aggregation.

***Proximal preferential attachment* (PPA):** Algorithm 2 was motivated by our work in milestone 1, in which we found that a fixed scale-free topology including an intermediate aggregation at the "hub devices" was equally effective as centralized FL for devices with fixed positions. In our setup, the server would run this algorithm during the "linking phase". PPA would perform the standard preferential attachment with an added caveat that each device d could

Algorithm 2 Proximal Preference Attachment (PPA)

Inputs: $k, \mathcal{X}, d(\cdot,\cdot) \leftarrow$ num seeds, devices, dist metric
Init empty $\mathcal{G}(V, E)$
Define: $\rho(x_i, x_j) := {}^{deg(x_j)}/_{deg(x_i)}$
Seed k Nodes into $\mathcal{G}$ and define $\mathcal{Y} \leftarrow \mathcal{X} \setminus \mathcal{G}$
for $x_i \in \mathcal{Y}$ **do**
  $\mathcal{G} \leftarrow x_i$
  $\Pi \leftarrow \pi_j = \rho(x_i, x_j) \sum_{k \in \mathcal{G}} \sum_{l \in \mathcal{G}} \rho(x_k, x_l) \forall x_j \in G$
  for k rounds **do**
    $x_j \leftarrow G_\Pi$ Sample from G with distribution Π
    $E \leftarrow e_{ij}$ add new edge to $\mathcal{G}$
  end for
end for
return $\mathcal{G}$

only attach to certain number of neighboring devices n in a given vicinity. Detailed in algorithm 2

Cosine Reassignment: Reassignment is motivated by the idea that, in a non-iid setting, we group devices with similar data distributions. For example, devices whose datasets contain a high percentage of dog images can aggregate at the same access point (AP), enabling them to share relevant knowledge more efficiently. We achieve this self-organization indirectly through information contained in model weights. Specifically, each device stores a copy of its community model and calculates the cosine similarity between its own model and the community model. Devices also compare their models to those of *neighboring nodes*, allowing them to reassign themselves to the most suitable community. The algorithm is outlined in algorithm 3. Notably, when running this algorithm independently, we base the topology on a *proximity threshold*(ϵ) instead of using a preferential attachment mechanism.

Algorithm 3 Local Reassignment

Inputs: N Devices (V), Similarity Metric $f(\cdot,\cdot)$, Model Weights $\boldsymbol{w_i}$, Threshold ϵ , Communities $\boldsymbol{C}_t$
Make Proximity Graph $G_t(V, E)$
where $E := \{e_{ij} | x_i, x_j \in V \text{ if } d(x_i, x_j) < \epsilon\}$
for $x \in G_t$ **do**
  $\mathcal{S}(x) \leftarrow \mathcal{N}_t(x) \cup \{x\}$
  $s_{min} \leftarrow \min_s \{f(\boldsymbol{w}^x, \boldsymbol{w}^s_{AP});\ \forall s \in \mathcal{S}(x)\}$
  $C_{t+1} \leftarrow (AP(s_{min}) \rightarrow AP(x))$ Assign x to most similar community
end for
return C_{t+1} (New parent map)

Dynamic Proximal Preference (DPP): The DPP algorithm extends the PPA and cosine reassignment algorithms (Algorithms 2 and 3). It constructs a graph in the same way as the PPA does in Algorithm 2, but with a slight modification to the weights Π. Specifically, the weights are adjusted to π_n (shown in algorithm 4), where $\cos_{sim}(d, n)$ represents the cosine similarity between devices d's model and n's community model. This metric is incorporated into the weights of the preferential attachment to induce a local assignment among devices that share similar data distributions based on their community model similarity. This approach helps to cluster similar devices, and deviate from long-range links; visualized in fig. 2. Thus improving model training efficiency and overall network performance.

Algorithm 4 Weighted-Preference Attachment

$D \leftarrow$ list of devices
for each device d in D **do**
  $\Pi \leftarrow$ empty list of weights
  for each proximity neighbor n of d **do**
    $\Pi \leftarrow \pi_n = \frac{\text{Degree}(n) + 5 \cdot \cos_{\text{sim}}(d,n)}{\text{Dist}(d,n)}$
  end for
  Attach k edges between d and neighbors sampled from Π
end for
return

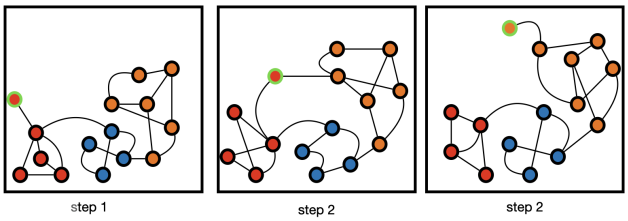

Figure 2: Weighted Proximal Preferential Attachment in mitigating an isolated node. In **Step 1**, the node is assigned to the red community due to both proximity and model similarity. **Step2**, the node is in between the red and orange community, however due to model similarity from previous step, we would want it to be a part of the red community. **Step 3**, given the distance the node is from the red community, in order to save in network cost, we would like to have them be a part of the orange community.

Divisive Modularity Algorithm: Motivated by some of our observations in Milestone 2, we tried one last modification of DPP which would also further exploit the hierarchical scheme. This modification adds one additional level to the FL hierarchy, by dividing each of the 5 communities further into 2 new communities (divisive step). This allowed us to focus on proximity at the first level, where we use k-means to group devices first on proximity, then at the second step we run a more isolated/targeted version of DPP to encourage a second formation of communities based on cosine similarities. Our hypothesis was that this would allow us to reduce network cost even further while hopefully retaining the advantages of using model similarities which helped performance of DPP. However,

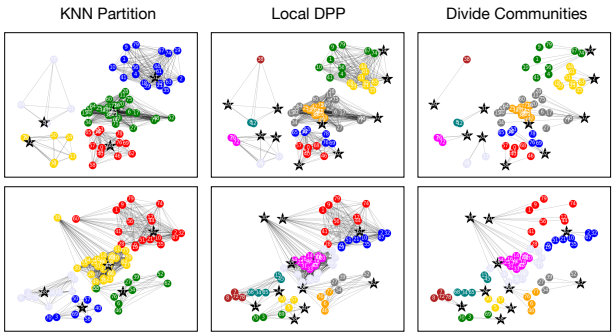

Figure 3: Our concept for divisive DPP. We begin by creating localized clusters with K-Nearest Neighbors. In the middle panel, we run DPP *only* on these clusters. Finally, we use Kernighan-Lin to divide each cluster into two new communities where edges are DPP scores

this sometimes led to communities that were too small and likely ended up hurting overall global performance. This process is visualized in fig. 3

We think there is more to be investigated here and that several things might be included to help this algorithm perform better. For instance, with only 100 devices, some of the 10 communities can end up having much fewer devices involved than others. This likely hurts performance when aggregated back into the global model more than using DPP grouping helps. Also, perhaps it is sometimes better for a community not to undergo the divisive step (if it is already quite small). However, we leave these questions to our future work.

3.4 Loss Optimization

To further enhance both the stability and convergence speed of our algorithm, we explored two alternative loss optimization strategies: FedProx [5] and FedMax [2]. Both approaches build upon the foundational FedAvg method, but introduce distinct modifications. FedProx incorporates a regularization term, representing the discrepancy between global and locally trained model weights, directly into the cross-entropy loss. In contrast, FedMax shifts the focus from parameter differences to the activation space of the model by maximizing the entropy of the activation vectors. These methodological adjustments aim to expedite convergence, improve stability, and ultimately yield a more robust federated learning performance. See fig. 5.

4 Experiment

4.1 Setup

We run each of the four algorithms (star, PPA, reassignment, DPP) for 35 global rounds with a stopping condition for 95% (spoiler: none of them reach this). We initialize the 100 devices with a subset of images from the CIFAR-10 dataset. To obtain a non-iid distribution we used a Dirichlet partitioning. Then we simulate device mobility, essentially running algorithm 1 for a total of $T_S = 35$ rounds for each of our algorithms as well as the star baseline.

4.2 Relevant Results

We compare the head-to-head accuracies evaluated at each global round, which is plotted in fig. 4. Network cost is calculated as the sum of all edge weights multiplied by the number of communications over that edge and wall clock time is the recorded total time it takes to run each algorithm. These costs are recorded and shown in **??**. Furthermore, table 1 showcases

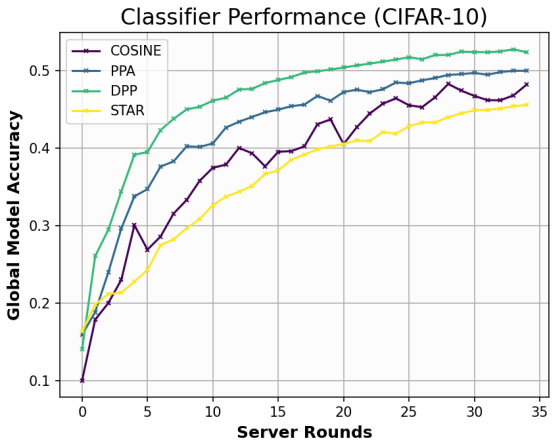

Figure 4: Global Comparison Plot: We see the four mentioned algorithms and their training progress on CIFAR overlaid above. All semi-decentralized algorithms train faster then the centralized baseline in terms of global server rounds. PPA's performance shows that adding the aggregation step at hub devices can beat the star baseline. While DPP shows that we can get the best performance by combining both reassignment and preferential attachment.

	Wall Clock Time[s]	**Network Cost**
Star	691.22	3139.5
PPA	763.48	2938.02
Cosine	2497.14	146,744.5
DPP	402.75	1710.7
MaxDPP	**431.66**	**1602.064**

Table 1: Costs to Reach 45.56% Accuracy

4.3 Implications

Overall, our semi-decentralized approaches outperform the centralized federated learning (FL) baseline (STAR) by achieving higher accuracy in fewer global rounds. For example, fig. 4 shows that both PPA (blue line) and DPP (green line) converge more rapidly than STAR (yellow line) and COSINE (purple line), demonstrating the benefit of incorporating hub-based aggregation and preferential attach-

ment. Despite these improvements in training speed, our methods have yet to surpass 55% accuracy under the current experiment configurations.

Network Costs: Moreover, as table 1 illustrates, DPP and MaxDPP (DPP with FedMax loss scheme)offer a more balanced outcome, both reaching the desired accuracy level faster and with less communication overhead than the other methods. MaxDPP, in particular, achieves the lowest time and cost, indicating that its combination of selective reassignment and preferential attachment provides a more resource-efficient training process. In summary, this table highlights how thoughtful network structure and adaptive reassignment strategies can significantly reduce both the temporal and communication burdens of federated learning.

The jagged training curves observed when using only local community reassignment 3 suggest that this process can sometimes degrade model performance. Such temporary setbacks likely stem from the stochastic nature of device assignments. If devices are reassigned to less compatible communities, the collective model may momentarily "unlearn" previously integrated knowledge. Conversely, integrating cosine-based weighting into the community assignment process (as in DPP) helps achieve more stable improvement. However, this stability comes at a higher cost, especially when applied to a densely connected network without initial preferential attachment, as reflected in Cosine's significantly increased network overhead shown in table 1.

Looking more closely at the network evolution (fig. 1), we observe that DPP's flexibility allows it to shift between configurations favoring a few dominant hubs and more evenly distributed structures. This adaptive behavior suggests that there may be considerable room to further refine the topology—by making it sparser and more localized. Such refinements could potentially reduce communication costs, improve model accuracy, and maintain the rapid convergence we see with the semi-decentralized approaches.

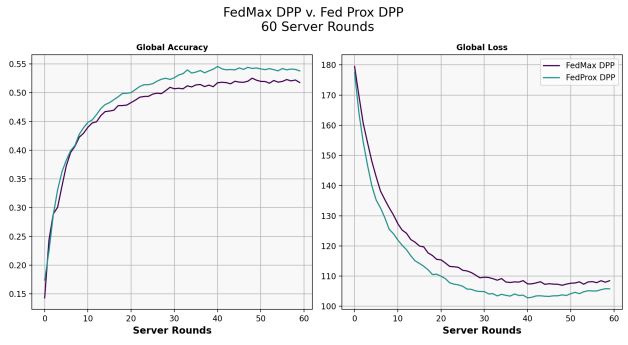

Figure 5: DPP algorithm trained on CIFAR10 non-IID for 60 global communication rounds with different optimization schemes FedProx and FedMax

In comparing different loss optimization strategies, such as FedProx and FedMax, we focus our analysis on the DPP approach, as it yields the strongest overall performance. As illustrated in fig. 5, FedProx and FedMax converge at a similar rate; however, FedProx consistently outperforms FedMax by achieving higher global accuracy and lower global loss. Interestingly, toward the later rounds of training, both methods exhibit a slight increase in loss. This upward trend suggests the onset of client drift, where local models begin to overfit to their non-IID training distributions. As a result, the global model struggles to maintain generalization performance, highlighting the importance of mitigating client drift in federated learning frameworks.

Furthermore, fig. 6 displays in the top row, MODDPP maintains significantly lower communication cost than all other methods, while STAR and COSINE incur the highest costs. The runtime plot shows that COSINE runs slowest, while the other semi-decentralized methods remain more efficient. In the bottom row, these metrics are normalized per unit accuracy gained, further illustrating that MODDPP and other hierarchical approaches can achieve better accuracy efficiency with considerably less overhead compared to the baseline methods.

5 Conclusion

In this work, we investigated the design and performance of novel hierarchical and adaptive federated learning schemes in a dynamic, mobility-driven environment. Our goal was to improve upon the baseline centralized (star) topology by incorporating intermediate, device-level aggregation points and leveraging strategies inspired by preferential attachment and model-based community reassignment. Through experiments on CIFAR-10 with non-iid data distributions and a realistic mobility dataset, we observed that these semi-decentralized approaches can achieve faster convergence in fewer global rounds compared to the centralized baseline, highlighting the value of distributed network structures and strategic community formation.

Our results show that preferential attachment-based schemes (PPA and DPP) accelerate training while maintaining reduced communication overhead. The more refined DPP approach, which integrates model similarity metrics, further improved both performance and stability. Meanwhile, community reassignment offered valuable flexibility but required careful management to avoid introducing instability or increased costs due to excessive network complexity. MaxDPP, an extension of these ideas, demonstrated a promising balance between speed, accuracy, and communication efficiency.

Overall, our findings suggest that intelligently combining hierarchical aggregation, adaptive topology selection, and local model similarity can better harness device mobility and heterogeneous network conditions for more efficient

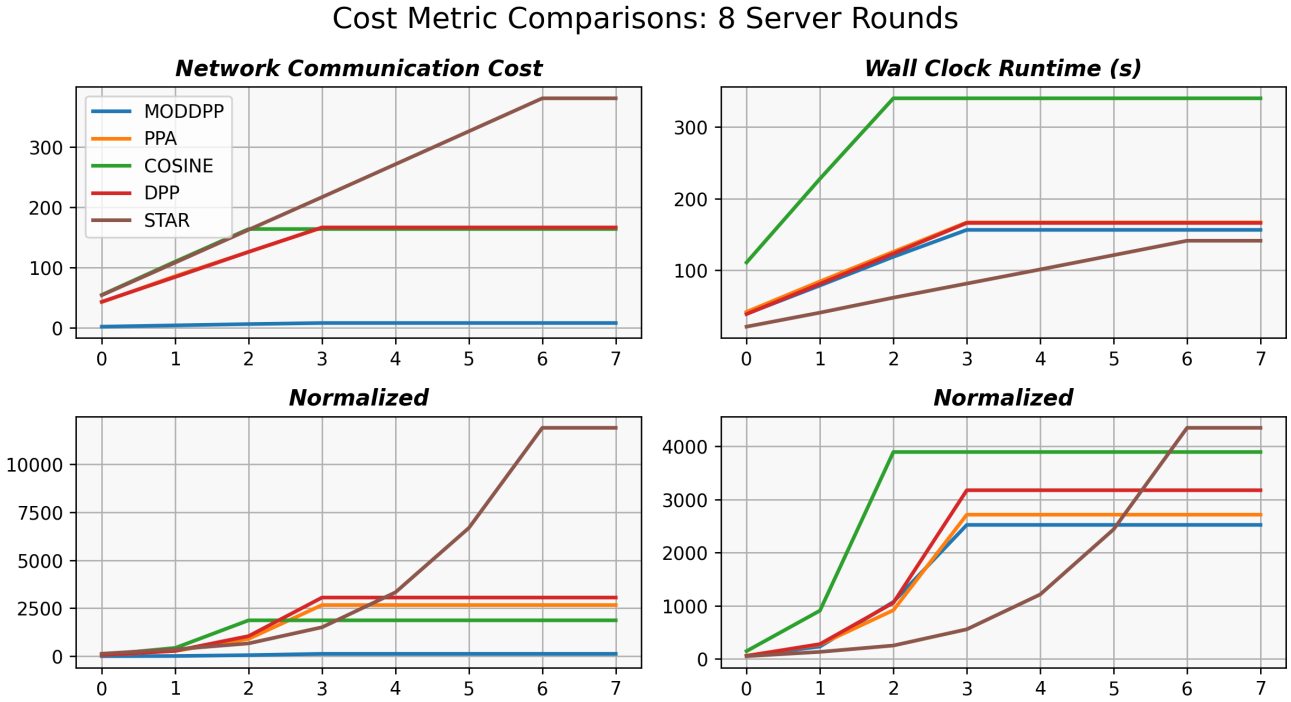

Figure 6: Network Comparison Plot: compares the network communication cost and the wall clock runtime of 4 algorithms over eight global server rounds on the MNIST dataset, along with their normalized values relative to accuracy gains in the second row. This shows our algorithms are more efficient in terms of accuracy gains for equivalent cost (in both runtime and network). Note that Star also requires the highest overall network cost here.

federated learning. While we have not yet surpassed moderate accuracy thresholds under our current configuration, these results open the door for future work.

Future Considerations: Potential avenues include refining hierarchical structures to mitigate overhead, exploring more robust approaches to community formation, more regularization to mitigate client drift, and incorporating adaptive optimization methods tailored to dynamic network conditions. We also believe the self-organization process can be enhanced by enabling devices to remember and favor past community memberships that were beneficial for training. This approach would lead to more stable assignments, more personalization, and further reduced costs. Ultimately, our study provides a foundation for the development of scalable, resource-efficient, and adaptive federated learning frameworks suitable for ever-changing, real-world environments.

Contributions: Both members contributed to ideation, implementation, reporting, figures, and presentation. Jacob focused on setting up distributed training/testing loops, configuring experiments, and preparing datasets and dataloaders for MNIST&CIFAR. Nick concentrated on integrating custom algorithms with the training/testing procedure, setting up mobility data, and mapping device locations to NetworkX graphs. Both collaborated on plotting, reporting, and custom algorithm ideation.

References

[1] A. Barrat, M. Barthélemy, and A. Vespignani. Weighted evolving networks: coupling topology and weights dynamics. *arXiv preprint cond-mat/0401057*, 2004.

[2] W. Chen, K. Bhardwaj, and R. Marculescu. Fedmax: Mitigating activation divergence for accurate and communication-efficient federated learning. In *Proceedings of the Federated Learning Workshop*, 2020. Available at https://arxiv.org/abs/2004.03657.

[3] L. Chou, Z. Liu, Z. Wang, and A. Shrivastava. Efficient and less centralized federated learning. In *Machine Learning and Knowledge Discovery in Databases. Research Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part I 21*, pages 772–787. Springer, 2021.

[4] A.-J. Farcas et al. Mohawk: Mobility and heterogeneity-aware dynamic community selection for hierarchical federated learning. In *Proceedings of the International Conference on Internet-of-Things Design and Implementation (IoTDI '23)*, pages 1–13, San Antonio, TX, USA, 2023. ACM.

[5] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. *arXiv preprint arXiv:1812.06127v5*, 2020.

[6] L. Palmieri et al. The effect of network topologies on fully decentralized learning: A preliminary investigation. In *Proceedings of the 1st International Workshop on Networked AI Sys-*

tems (NetAISys '23), pages 1–6, New York, NY, USA, June 2023. ACM.